

Lab 05

Software Verification and Validation Lab

SWE 4806



Instructor: Talimul Bari Shreshtho

Date: July 31, 2026



Lab 06: Integration Testing and Continuous Integration

Objective

The objective of this lab is to extend the calculator project developed in the previous lab into a simple multi-page web application. Students will integrate multiple software modules, develop integration tests, demonstrate the use of **stubs** or **drivers**, and automate testing using GitHub Actions.

Prerequisite

You should already have completed the previous laboratory assignment containing:

- Basic Calculator module
- Advanced Calculator module
- Unit tests using Jest

The existing calculator logic should be reused in this lab.

Project Description

Develop a web-based calculator application consisting of four pages.

- a) Home Page
- b) Basic Calculator
- c) Advanced Calculator
- d) Custom Calculator

The Home Page shall provide navigation to all calculator pages.

The Basic and Advanced Calculator pages should reuse the calculator modules developed in the previous lab.

The Custom Calculator must implement a real-world calculation by integrating both the Basic Calculator module and the Advanced Calculator module.

Suggested Project Structure

calculator-app/

```
index.html
basic.html
advanced.html
custom.html
```

| Scripts/



```
basic.js
advanced.js
custom.js
```

Tests/

```
basic.test.js
advanced.test.js
integration.test.js
```

```
.github/
  workflows/
    node.yml
```

```
package.json
```

Task 1: Develop the Website

Create the following web pages.

- Home Page
- Basic Calculator
- Advanced Calculator
- Custom Calculator

The Home Page should contain navigation buttons that redirect users to each calculator.

Task 2: Implement a Custom Calculator

Develop one real-world calculator that combines the functionality of both calculator modules.

Examples include:

- Scientific Calculator
- Loan Calculator
- Compound Interest Calculator
- Shopping Bill Calculator
- Electricity Bill Calculator
- Investment Calculator

The implementation must invoke functions from both:

- `basic.js`
- `advanced.js`



Task 3: Testing

Extend the existing test suite from the previous laboratory to verify the newly developed Custom Calculator.

- a) Write **Unit Tests** for all new functions implemented in the Custom Calculator module.
- b) Write **Integration Tests** to verify that the Custom Calculator correctly interacts with both the Basic Calculator module (`basic.js`) and the Advanced Calculator module (`advanced.js`).
- c) Demonstrate one integration testing technique by implementing either:
 - A **Stub** that replaces the Basic or Advanced Calculator module during testing, or
 - A **Driver** that invokes and tests the Custom Calculator module independently before integrating it with the web application.

Your test suite should verify:

- Correct functionality of the Custom Calculator.
- Correct interaction between the Basic and Advanced Calculator modules.
- Proper handling of invalid and boundary inputs.
- Correct behaviour when using the implemented Stub or Driver.

Task 4: Continuous Integration

Push the project to GitHub.

Create a GitHub Actions workflow that automatically performs the following steps whenever code is pushed to the repository.

- a) Checkout the repository
- b) Install Node.js
- c) Install project dependencies
- d) Execute all Unit Tests
- e) Execute all Integration Tests

The build should fail if any test case fails.

Submission Requirements

Submit the following:

- GitHub Repository Link



Appendix: Useful Jest Features for Integration Testing

Jest provides several built-in features that are useful for writing integration tests. The following example demonstrates how to verify that multiple modules interact correctly during program execution.

Example: Verifying Module Interaction

Assume that the `Custom Calculator` internally uses both the `basic.js` and `advanced.js` modules.

```
const basic = require("../Scripts/basic");
const advanced = require("../Scripts/advanced");
const custom = require("../Scripts/custom");

describe("Integration Tests", () => {

  test("Compound Interest should use both basic and advanced modules", () => {

    const addSpy = jest.spyOn(basic, "add");
    const divideSpy = jest.spyOn(basic, "divide");
    const multiplySpy = jest.spyOn(basic, "multiply");
    const subtractSpy = jest.spyOn(basic, "subtract");

    const powSpy = jest.spyOn(advanced, "pow");

    custom.compoundInterest(1000, 10, 2);

    expect(divideSpy).toHaveBeenCalled();
    expect(addSpy).toHaveBeenCalled();
    expect(powSpy).toHaveBeenCalled();
    expect(multiplySpy).toHaveBeenCalled();
    expect(subtractSpy).toHaveBeenCalled();

    addSpy.mockRestore();
    divideSpy.mockRestore();
    multiplySpy.mockRestore();
    subtractSpy.mockRestore();
    powSpy.mockRestore();
  });
});
```



Explanation

- **jest.spyOn(object, method)** creates a **Spy** that monitors an existing function without changing its original implementation.
- When the Custom Calculator executes, the Spy records every invocation of the selected function.
- **expect(spy).toHaveBeenCalled()** verifies that the function was invoked during execution.
- **mockRestore()** removes the Spy and restores the original function. This prevents one test from affecting another.

Useful Jest Assertions

Some commonly used Jest assertions are shown below.

Assertion	Purpose
toBe(value)	Checks exact equality.
toEqual(value)	Compares objects and arrays.
toBeCloseTo(value)	Compares floating-point numbers.
toHaveBeenCalled()	Verifies that a function was called.
toHaveBeenCalledTimes(n)	Checks how many times a function was called.
toHaveBeenCalledWith(args)	Checks the arguments passed to a function.
toThrow()	Verifies that a function throws an exception.

When Should a Spy Be Used?

A Spy is useful when the objective of the test is not only to verify the final result but also to verify that different modules interact correctly.

For example, if the specification requires the Custom Calculator to reuse functions from `basic.js` and `advanced.js`, a Spy can verify that these functions were actually invoked instead of reimplementing the same logic inside the Custom Calculator.